



10. Multiprocessor Scheduling

Intro

Single CPU with cache

Cache coherence (캐시 일관성)

Cache coherence solution (해결책)

Bus snooping

Don't Forget Synchronization

Cache Affinity (캐시 친화성)

Single queue Multiprocessor Scheduling (SQMS)

Multi-queue Multiprocessor Scheduling (MQMS)

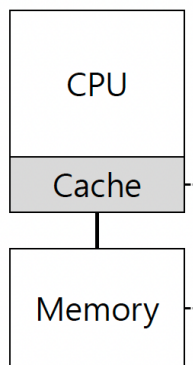
Load Imbalance issue of MQMS

Work Stealing

Intro

- 멀티코어 프로세서의 등장은 멀티프로세서 스케줄링의 확산을 가져옴
 - **Multicore**: 단일 Chip에 여러 개의 CPU Cores
- CPU를 더 추가한다고 해서 단일 애플리케이션의 실행 속도가 빨라지는 것은 X
→ 스레드를 사용하여 애플리케이션을 병렬로 실행하도록 다시 작성해야 함

Single CPU with cache



Cache

- 작고 빠른 메모리
- Main memory에서 자주 사용하는 **Popular Data**를 복사해놓음
- Utilize **temporal** and **spatial** locality

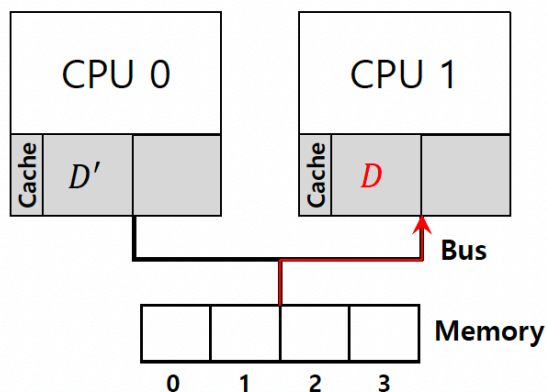
Main Memory

- Holds all of the data
- Cache보다 접근이 느림

- 느린 메인 메모리가 빠른 것처럼 보이게 하는 효과가 있음

Cache coherence (캐시 일관성)

- Consistency of shared resource data stored in multiple caches



- 위에 있는 게 private cache
- CPU1 은 correct value D' 대신에 old value D 에 접근함

Cache coherence solution (해결책)

Bus snooping

- 각 cpu core의 Cache가 Bus에 메모리 업데이트가 있는지 계속 확인
 - HW적으로 구현
- CPU가 자신의 Cache에 있는 data의 update를 발견하면...

- 복사본을 **update** 하거나 **invalidate**

1. Write-Update

- a. 내가 값을 수정하면 → "**야 새 값은 100이야!**" 하고 다른 캐시들한테 **직접 새 값을 보내줌**
- b. 다른 캐시들은 그냥 받아서 자기 값을 업데이트하면 끝
- c. Overhead 큼

2. Write-Invalidate

- a. 내가 값을 수정하면 → "**야 그 값 무효야! 버려!**" 하고 무효화 신호만 보냄
- b. 다른 캐시들은 자기 값을 invalidate 상태로 바꿈
- c. 나중에 그 친구(다른 CPU)가 그 값이 필요하면 → **나(나의 캐시) 또는 메모리에 다시 요청해서 가져옴**

Don't Forget Synchronization

- Application Program이 Shared Data를 접근할 때 Lock & Unlock을 통한 동기화가 필요

```
pthread_mutex_t m;
typedef struct __Node_t {
    int value;
    struct __Node_t *next;
}
Node_t;

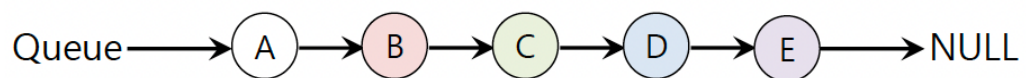
int List_Pop() {
    lock(&m);
    Node_t *tmp= head; // remember old head ...
    int value = head->value; // ... and its value
    head = head->next; // advance head to next pointer
    free(tmp); // free old head
    unlock(&m);
    return value; // return value at head
}
```

Cache Affinity (캐시 친화성)

- 가능하면 한 process를 **동일한 CPU**에 유지시켜라
 - 프로세스가 CPU의 캐시에 꽤 많은 상태(데이터)를 쌓아둔다.
 - 다음에 그 프로세스가 실행될 때, 만약 그 CPU의 캐시에 이미 데이터가 남아있으면 더 빠르게 실행
- 멀티프로세서 스케줄러는 스케줄링 결정을 할 때 캐시 친화성(affinity)을 고려해야 함.

Single queue Multiprocessor Scheduling (SQMS)

- Schedule의 대상이 되는 모든 Job을 하나의 Queue에 집어넣어
 - 각 CPU는 단순히 그냥 globally shared queue에서 다음 수행할 job 뽑아옴
 - 단점
 - **locking** 필요함. 왜? shared data structure니까 → Lack of scalability(확장성의 한계)
 - 동기화 작업의 필요성으로 인해 락 경쟁 발생 → CPU 수 증가할 수록 락 대기 시간 길어짐 → 확장성 한계 존재
 - **Cache affinity**
 - Example



- Possible job scheduler across CPUs

CPU0	A	E	D	C	B	... (repeat) ...
CPU1	B	A	E	D	C	... (repeat) ...
CPU2	C	B	A	E	D	... (repeat) ...
CPU3	D	C	B	A	E	... (repeat) ...

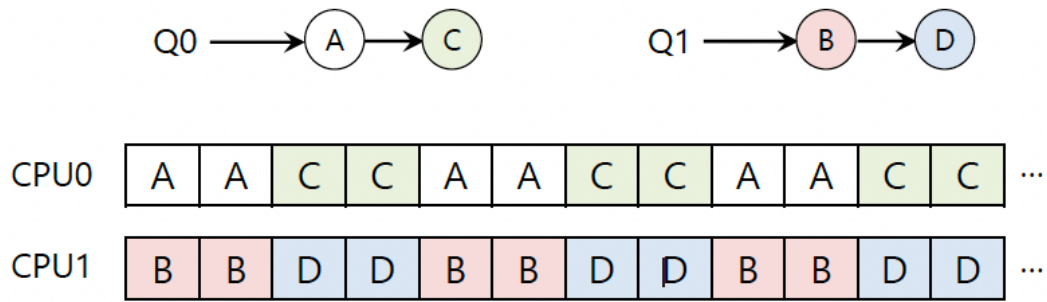
▪ **Preserving affinity** for most

- 한 **Process**를 계속해서 동일한 **CPU**에서 돌려보자!
- Jobs A ~ D는 움직이지 X
- Job E만이 Migrating from CPU to CPU.
 - 아까 전보다는 우수함. 이렇게 하면 되긴 하는데 복잡함

CPU0	A	E	A	A	A	... (repeat) ...
CPU1	B	B	E	B	B	... (repeat) ...
CPU2	C	C	C	E	C	... (repeat) ...
CPU3	D	D	D	D	E	... (repeat) ...

Multi-queue Multiprocessor Scheduling (MQMS)

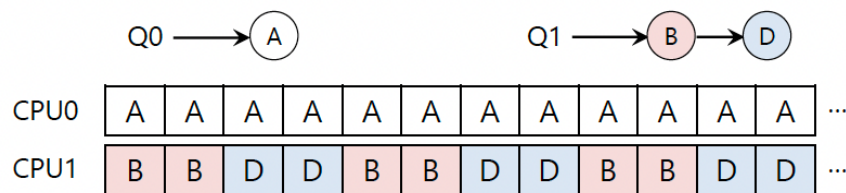
- 여러 개의 scheduling queues.
 - 각 queue마다 particular scheduling discipline.
 - job은 정확히 임의의 한 queue로
 - information sharing & synchronization 문제 해결
- With **round robin**, the system might produce a schedule that looks like this:



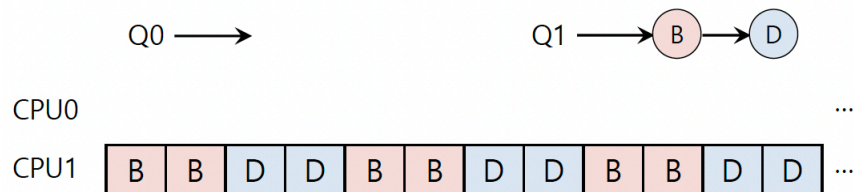
MQMS provides more **scalability** and **cache affinity**

Load Imbalance issue of MQMS

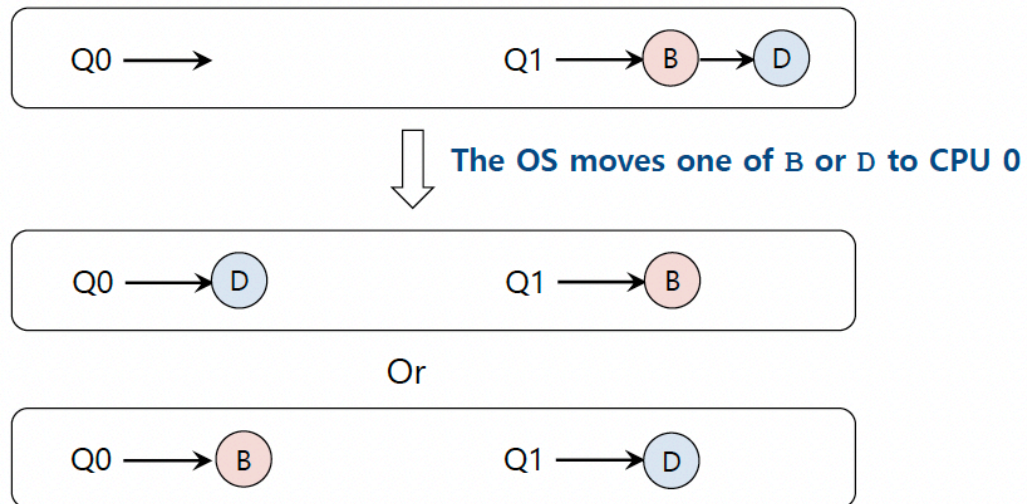
- 앞선 예시에서 Q0의 C가 끝났어



- A는 B와 D CPU time의 2배
- 이후 Q0의 A가 끝났어



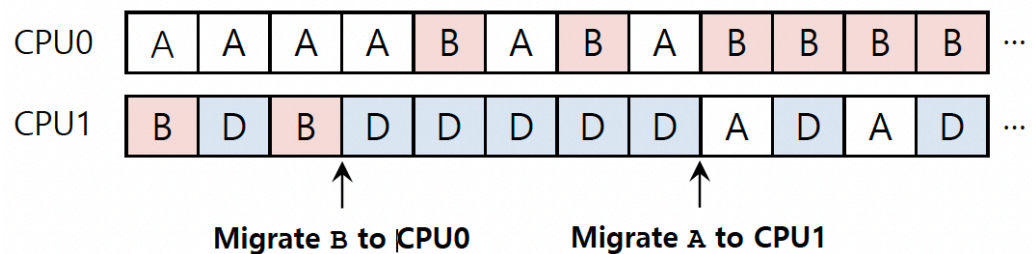
- CPU0는 idle한 상태
- 해결책은 job을 옮겨! → **Migration**
 - Example
 - B를 옮기든지 D를 옮기든지 해라



- A more tricky case



- A possible migration pattern:
 - Keep switching jobs - 서로 교환 계속



Work Stealing

- queue 간 job 이동
 - 구현
 - 우선 source queue(job이 적은)를 선정
 - Source queue가 가끔씩 다른 target queue를 슬쩍 확인
 - 만약 target queue가 source queue보다 더 많이 차있으면, source가 target에서 작업 하나 이상을 "Steal"

- 단점
 - 높은 overhead & 확장을 하는 데 문제가 있음
- work stealing은 코디네이터가 없다?
- 뺏기는 녀석에 대해 race가 발생할 수 있다. → scalability의 한계가 있다